

Chapter 8. Basic Principles

From the four values we derive a dozen or so basic principles to guide our new style. We will check proposed development practices to see how they measure up to these principles.

"Learning to Drive" reminds us to make lots of small changes and never take our eyes off the road. The four values—communication, simplicity, feedback, and courage—give us our criteria for a successful solution. However, the values are too vague to give us much help in deciding which practices to use. We need to distill the values into concrete principles we can use.

These principles will help us as we choose between alternatives. We will prefer an alternative that meets the principles more fully to one that doesn't. Each principle embodies the values. A value may be vague. One person's simple is another person's complex. A principle is more concrete. Either you have rapid feedback or you don't. Here are the fundamental principles:

- Rapid feedback
- Assume simplicity
- Incremental change
- Embracing change
- Quality work

Rapid feedback—Learning psychology teaches that the time between an action and its feedback is critical to learning. Animal experiments show that even small differences in the timing of feedback result in large differences in learning. A few seconds delay between stimulus and response and the mouse doesn't learn that the red button means food. So, one of the principles is to get feedback, interpret it, and put what is learned back into the system as quickly as possible. The business learns how the system can best contribute, and feeds back that learning in days or weeks instead of months or years. The programmers learn how best to design, implement and test the system, and feed back that learning in seconds or minutes instead of days, weeks, or months.

Assume simplicity—Treat every problem as if it can be solved with ridiculous simplicity. The time you save on the 98% of problems for which this is true will give you ridiculous resources to apply to the other 2%. In many ways, this is the hardest principle for programmers to swallow. We are traditionally told to plan for the future, to design for reuse. Instead, XP says to do a good job (tests, refactoring, communication) of solving today's job today and trust your ability to add complexity in the future where you need it. The economics of software as options favor this approach.

Incremental change—Big changes made all at once just don't work. Even in Switzerland, center of meticulous planning, where I live now, they don't try to make big changes. Any problem is solved with a series of the smallest changes that make a difference.

You'll find incremental change applied many ways in XP. The design changes a little at a time. The plan changes a little at a time. The team changes a little at a time. Even the adoption of XP must be taken in little steps.

Embracing change—The best strategy is the one that preserves the most options while actually solving your most pressing problem.

Quality work—Nobody likes working sloppy. Everybody likes doing a good job. Of the four project development variables—scope, cost, time, and quality—quality isn't really a free variable. The only possible values are "excellent" and "insanely excellent," depending on whether lives are at stake or not. Otherwise you don't enjoy your work, you don't work well, and the project goes down the drain.

Here are some less central principles. They still will help us decide what to do in specific situations.

- Teach learning
- Small initial investment
- Play to win
- Concrete experiments
- Open, honest communication
- Work with people's instincts, not against them
- Accepted responsibility
- Local adaptation
- Travel light
- Honest measurement

Teach learning—Rather than make a bunch of doctrinaire statements like "Thou must do testing like XYZ," we will focus on teaching strategies for learning how much testing you should do. Also how much design, and refactoring, and everything else you should do. Some ideas that we will state with certainty. There will be others that we don't have quite so much confidence in, and those we will state as strategies with which the reader can learn their own answers.

Small initial investment—Too many resources too early in a project is a recipe for disaster. Tight

budgets force programmers and customers to pare requirements and approaches. The focus a tight budget generates encourages you to do a good job of everything you do. However, resources can be too tight. If you don't have the resources to solve even one interesting problem, then the system you create is guaranteed not to be interesting. If you have someone dictating scope, dates, quality, and cost, then you are unlikely to be able to navigate to a successful conclusion. Mostly, though, everyone can get by with fewer resources than they are comfortable with.

Play to win—It was always wonderful to watch John Wooden's UCLA basketball teams. They were usually crushing the opposition. However, even if the game was close going into the final minutes, UCLA was absolutely certain that they were going to win. After all, they had won so many, many times before. So, they were relaxed. They did what they needed to do. And they won again.

I remember an Oregon basketball game that provides a stark contrast. Oregon was playing a nationally ranked Arizona team that was destined to send four players to the NBA. At halftime, amazingly, Oregon was up by 12 points. Arizona couldn't do anything right, and Oregon's offense had them baffled. After the half, however, Oregon came out and tried to play as slowly as possible to reduce the number of points scored and preserve the victory. The strategy didn't work, of course. Arizona found ways to use their huge advantage in talent to win the game.

The difference is between playing to win and playing not to lose. Most software development I see is played not to lose. Lots of paper gets written. Lots of meetings are held. Everyone is trying to develop "by the book," not because it makes any particular sense, but because they want to be able to say at the end that it wasn't their fault, they were following the process. CYA.

Software development played to win does everything that helps the team to win and doesn't do anything that doesn't help to win.

Concrete experiments—Every time you make a decision and you don't test it, there is some probability that the decision is wrong. The more decisions you make, the more these risks compound. Therefore, the result of a design session should be a series of experiments addressing the questions raised during the session, not a finished design. The result of a discussion of requirements should also be a series of experiments. Every abstract decision should be tested.

Open, honest communication—This is such a motherhood statement that I almost left it out. Who wouldn't want to communicate openly and honestly? Everybody I visit, it seems. Programmers have to be able to explain the consequences of other people's decisions, "You violated encapsulation here and it really messed me up." They have to be able to tell each other where there are problems in the code. They have to be free to express their fears, and get support. They have to be free to deliver bad news to customers and management, to deliver it early, and not be punished.

If I see someone looking around to see who is listening before answering a question, I take it as a sign of deep trouble on the project. If there are personal matters to be discussed, I can understand the need for privacy. But which of two object models to use is not a matter that is helped by a "top secret" stamp.

Work with people's instincts, not against them—People like winning. People like learning. People

like interacting with other people. People like being part of a team. People like being in control. People like being trusted. People like doing a good job. People like having their software work.

Paul Chisolm writes:

I was in a meeting, and the quality assurance wannabe manager suggested adding about half a dozen fields (to an online form already full of data no one ever used), not because the information would be useful downstream, but allegedly because filling out these additional fields would SAVE TIME. My reaction: I banged my head against the conference room table, like a Warner Brothers cartoon character who's just heard something unbelievable, and told him to stop lying to me. (To this day, I don't know if that was one of the least professional things I ever did, or one of the most professional. I have been told by an eye doctor to stop banging my head against things; it can cause retinas to detach.) (Source: e-mail.)

It's been tricky, designing a process where following short-term self-interest also serves long-term team interest. You can expound all you want on how some practice or other is in everybody's best interest long-term, but when the pressure mounts, if the practice doesn't solve an immediate problem it will be discarded. If XP can't work with people's short-term interest, it is doomed to the outer methodological darkness.

Some folks really like this about XP, that it celebrates what programmers seem to do when left to their own devices, with just enough control to keep the whole process on track. One quote I remember was, "XP matches observations of programmers in the wild."

Accepted responsibility—No single action takes the life out of a team or a person more than being told what to do, especially if the job is clearly impossible. Primate dominance displays work only so long in getting people to act like they are going along. Along the way, a person told what to do will find a thousand ways of expressing their frustration, most of them to the detriment of the team and many of them to the detriment of the person.

The alternative is that responsibility be accepted, not given. This does not mean that you always do exactly what you feel like doing. You are part of a team, and if the team comes to the conclusion that a certain task needs doing, someone will choose to do it, no matter how odious.

Local adaptation—You have to adapt whatever you read in this book to your local conditions. This is an application of accepted responsibility to your development process. Adopting XP does not mean that I get to decide how you are going to develop. It means that you get to decide how to develop. I can tell you what I have found to work well. I can point out the consequences that I see from deviating. At the end of the day, however, this is your process. You have to decide on something today. You have to be aware of whether it still works tomorrow. You have to change and adapt. Don't read this thinking, "Finally, now I know how to develop." You should end up saying, "I have to decide all of this *and* program?" Yes, you do. But it's worth it.

Travel light— You can't expect to carry a lot of baggage and move fast. The artifacts we maintain should be:

- Few
- Simple
- Valuable

The XP team becomes intellectual nomads, always prepared to quickly pack up the tents and follow the herd. The herd in this case might be a design that wants to go a different direction than anticipated, or a customer that wants to go a different direction than anticipated, or a team member who leaves, or a technology that suddenly gets hot, or a business climate that shifts.

Like the nomads, the XP team gets used to traveling light. They don't carry much in the way of baggage except what they must have to keep producing value for the customer—tests and code.

Honest measurement—Our quest for control over software development has led us to measure, which is fine, but it has led us to measure at a level of detail that is not supported by our instruments. Better to say "This will take two weeks, more or less" than say, "14.176 hours," if you have no real way of estimating to this level of detail. We will also strive to choose metrics that have correlation to the way we want to work. For example, lines of code is a useless measurement in the face of code that shrinks when we learn better ways of programming.